PROGETTO SARRL – STAZIONE AMBIENTALE INTELLIGENTE

1. Introduzione

Questo progetto consiste nella realizzazione di una **stazione di monitoraggio ambientale intelligente** basata su un microcontrollore **ESP32**.

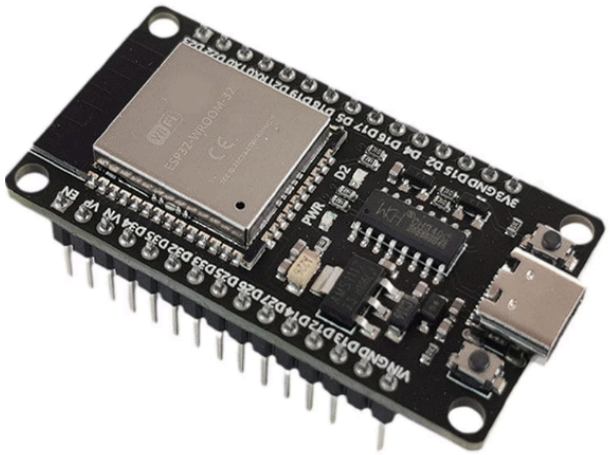
Il dispositivo è in grado di:

- Misurare diversi parametri ambientali.
- Visualizzare i dati su due schermi.
- Collegarsi a Internet tramite Wi-Fi.
- Inviare i dati a un'applicazione web tramite API.
- Ricevere comandi da remoto per controllare dispositivi (come una ventola).

In pratica il sistema funziona come una **piccola centralina ambientale domestica** capace di monitorare la qualità dell'ambiente e comunicare con un server online.

2. Componenti utilizzati

ESP32



L'**ESP32** è il componente principale del progetto.

È un **microcontrollore**, cioè un piccolo computer programmabile che contiene:

- processore
- memoria
- connessione Wi-Fi
- numerosi pin di ingresso e uscita

Questi pin permettono di collegare sensori, schermi e altri dispositivi elettronici.

Nel progetto l'ESP32 si occupa di:

- leggere i dati dai sensori
- controllare la ventola
- comunicare con il server web
- gestire gli schermi
- sincronizzare l'orario tramite internet

3. Sensori utilizzati

Sensori analogici e sensori digitali

Prima di descrivere i sensori utilizzati nel progetto è importante capire la differenza tra **sensori analogici** e **sensori digitali**.

Sensori analogici

Un **sensore analogico** restituisce un segnale elettrico continuo.

Questo significa che il valore della grandezza misurata viene rappresentato da una **tensione variabile**.

Esempio:

un sensore di temperatura analogico potrebbe restituire:

- 0.5 V → temperatura bassa
- 2.0 V → temperatura media
- 3.3 V → temperatura alta

Il microcontrollore deve quindi convertire questa tensione in un numero tramite un **convertitore analogico-digitale (ADC)**.

Sensori digitali

Un **sensore digitale** invece elabora già internamente il segnale e invia direttamente **numeri digitali** al microcontrollore.

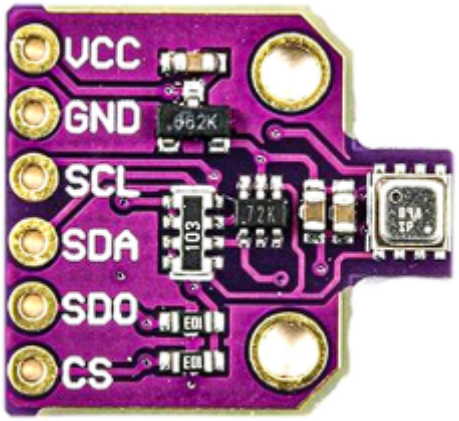
Questi dati vengono trasmessi tramite protocolli di comunicazione come:

- I2C
- SPI
- I2S

- UART

I sensori usati nel progetto sono **digitali**, quindi comunicano direttamente con l'ESP32 senza bisogno di conversioni analogiche.

Sensore BME680



Il **BME680** è un sensore ambientale digitale molto completo.

È in grado di misurare:

- temperatura
- umidità
- pressione atmosferica
- qualità dell'aria (gas)

Questo sensore utilizza un protocollo di comunicazione chiamato **I²C**, che permette di collegare più dispositivi usando solo due fili di comunicazione.

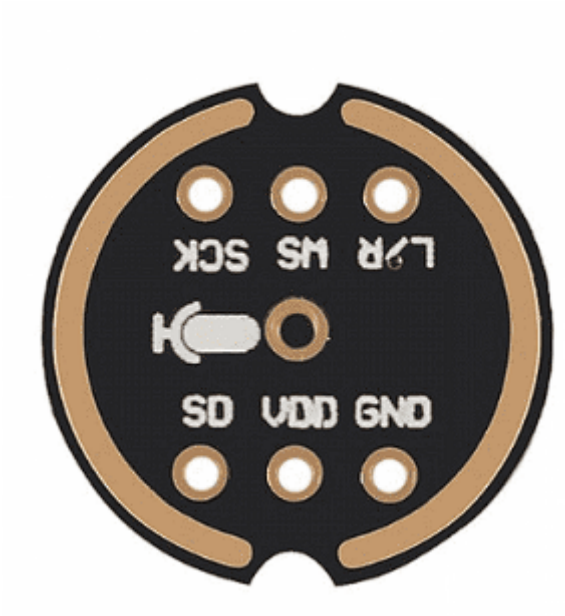
Sensore BH1750

Il **BH1750** è un sensore di luminosità digitale.

Misura la quantità di luce presente nell'ambiente e restituisce il valore in **lux**, l'unità di misura della luminosità.

Serve per capire quanto è illuminato l'ambiente in cui si trova il dispositivo.

Microfono INMP441



Il **INMP441** è un microfono digitale.

Permette di misurare il livello di rumore presente nell'ambiente.

A differenza degli altri sensori usa un protocollo chiamato **I2S**, utilizzato per trasmettere dati audio digitali.

Il microfono registra il suono e il programma lo trasforma in un valore espresso in **decibel (dB)**.

4. Sistema di raffreddamento e controllo della ventola

Nel progetto è stato integrato un sistema di raffreddamento composto da una **ventola a 5V** controllata automaticamente dall'ESP32.

L'obiettivo è permettere al dispositivo di **attivare una ventilazione quando la temperatura dell'ambiente supera una certa soglia**.

Nel programma questa soglia è impostata a **28°C**.

Perché non collegare direttamente la ventola all'ESP32

L'ESP32 possiede effettivamente un **pin di alimentazione a 5V** (generalmente chiamato **VIN** o **5V** sulla scheda).

Questo pin può alimentare dispositivi esterni.

Tuttavia collegare direttamente la ventola a un pin del microcontrollore **non è la soluzione più sicura o corretta**, per diversi motivi:

1. Corrente richiesta dalla ventola

Una ventola può richiedere **più corrente** rispetto a quella che un pin GPIO può fornire.

Un pin digitale dell'ESP32 può erogare circa:

20–40 mA

Una piccola ventola può richiedere anche:

100–200 mA o più

Questo potrebbe danneggiare il microcontrollore.

2. Protezione del microcontrollore

Le ventole sono **carichi induttivi**.

Quando vengono accese o spente possono generare **picchi di tensione**.

Questi picchi potrebbero:

- disturbare il funzionamento dell'ESP32
- danneggiare i pin del microcontrollore

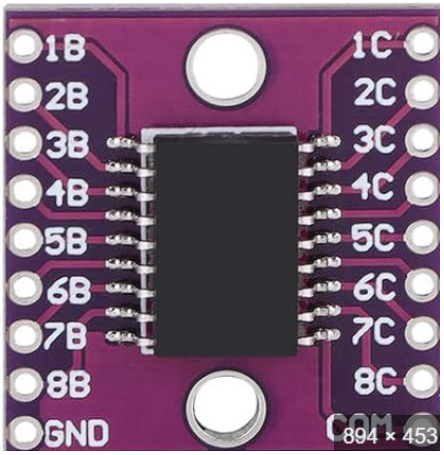
3. Controllo più affidabile

Utilizzare un circuito intermedio permette di:

- proteggere l'ESP32
- controllare dispositivi più potenti
- gestire più attuatori contemporaneamente

Per questo motivo nel progetto è stato utilizzato il **ULN2803A**.

Il componente ULN2803A



L'**ULN2803A** è un circuito integrato molto usato in elettronica per pilotare dispositivi che richiedono più corrente rispetto a quella che può fornire un microcontrollore.

È spesso utilizzato per controllare:

- motori
- relè
- lampade
- ventole
- solenoidi

Da cosa è composto

All'interno dell'ULN2803A sono presenti **8 transistor Darlington**.

Questo significa che il chip contiene **8 interruttori elettronici indipendenti**.

Ogni canale è composto da:

- **due transistor collegati in configurazione Darlington**
- **una resistenza di ingresso**
- **un diodo di protezione**

Configurazione Darlington

Una **coppia Darlington** è formata da **due transistor collegati insieme** per amplificare la corrente.

Il vantaggio è che:

- una **corrente molto piccola in ingresso**
- può controllare una **corrente molto più grande in uscita**

Questo permette all'ESP32 di controllare dispositivi che richiedono più potenza.

Il ruolo dei diodi di protezione

Ogni uscita dell'ULN2803A contiene un **diodo di protezione chiamato flyback diode**.

Questo diodo serve a proteggere il circuito dai picchi di tensione generati dai carichi induttivi.

Quando un motore o una ventola si spegne, il campo magnetico interno può generare una tensione inversa.

Il diodo devia questa energia evitando che raggiunga il microcontrollore.

Perché il ULN2803A è molto usato nei progetti embedded

Questo componente è molto diffuso perché offre diversi vantaggi:

- permette di controllare dispositivi ad alta corrente
- protegge il microcontrollore
- contiene già i diodi di protezione
- permette di controllare **fino a 8 dispositivi contemporaneamente**
- semplifica molto il circuito

Come funziona nel nostro progetto

Il funzionamento può essere semplificato così:

1. L'ESP32 invia un segnale digitale al pin di ingresso del ULN2803A.
2. Il transistor interno si attiva.
3. L'uscita del ULN2803A viene collegata a **massa (GND)**.
4. La corrente può quindi attraversare la ventola.
5. La ventola si accende.

In pratica il ULN2803A funziona come **un interruttore controllato elettronicamente**.

5. Schermi utilizzati

Il sistema utilizza **due display OLED**.

Display SH1107 (128x128)

Questo è lo schermo principale.

Mostra in tempo reale i dati dei sensori:

- temperatura
- umidità
- pressione
- qualità dell'aria
- luminosità
- rumore
- stato della ventola

Display SSD1306 (128x64)

Il secondo schermo è dedicato a:

- data
- ora

L'orario viene ottenuto automaticamente da internet tramite un servizio chiamato **NTP**.

6. Collegamento dei dispositivi

Per collegare i componenti viene utilizzata una **breadboard**.

Una breadboard è una basetta che permette di costruire circuiti elettronici senza saldature.

Alimentazione

Tutti i componenti devono essere collegati a:

- **3.3V** (alimentazione)
- **GND** (massa)

La massa è il punto di riferimento comune per tutto il circuito.

Cos'è un BUS di comunicazione

Un **bus** è un insieme di collegamenti elettrici condivisi da più dispositivi per scambiarsi dati.

Possiamo immaginarlo come **una strada sulla quale viaggiano le informazioni**.

Nel progetto i sensori comunicano con l'ESP32 tramite bus digitali.

I principali utilizzati sono:

- **I2C**
- **I2S**

Protocollo I2C

Il protocollo **I2C (Inter Integrated Circuit)** è uno dei sistemi più usati per collegare sensori ai microcontrollori.

La sua caratteristica principale è che utilizza **solo due fili**:

SDA → linea dati
SCL → linea di sincronizzazione

Su questo bus possono essere collegati **più dispositivi contemporaneamente**.

Ogni dispositivo possiede un **indirizzo univoco**.

Nel progetto abbiamo collegato:

- sensore **BME680**
- sensore **BH1750**
- display **SH1107**

tutti sullo stesso bus I2C.

Protocollo I2S

Il protocollo **I2S (Inter IC Sound)** è un sistema di comunicazione progettato specificamente per **trasmettere audio digitale**.

A differenza dell'I2C, utilizza **più linee di comunicazione**:

BCLK → clock del segnale audio
WS → sincronizzazione dei canali
SD → dati audio

Nel progetto questo protocollo viene usato dal **microfono INMP441** per inviare i dati audio all'ESP32.

Collegamento sensori (I2C)

I sensori BME680 e BH1750 usano il protocollo **I²C**.

Questo sistema utilizza due fili:

- SDA → dati
- SCL → sincronizzazione

Nel progetto sono collegati a:

- SDA → GPIO 21
- SCL → GPIO 22

Lo stesso bus I2C può essere condiviso da più dispositivi.

Collegamento microfono

Il microfono INMP441 utilizza I2S.

I collegamenti sono:

- WS → GPIO 27
- BCLK → GPIO 14
- SD → GPIO 12

Collegamento ventola

La ventola è collegata al driver ULN2803A.

Il driver riceve il segnale dal pin: GPIO 25

Nel progetto è stato utilizzato **un solo canale del ULN2803A**.

I collegamenti sono:

ESP32 GPIO25 → ingresso ULN2803A
uscita ULN2803A → negativo della ventola
positivo ventola → 5V
GND → massa comune

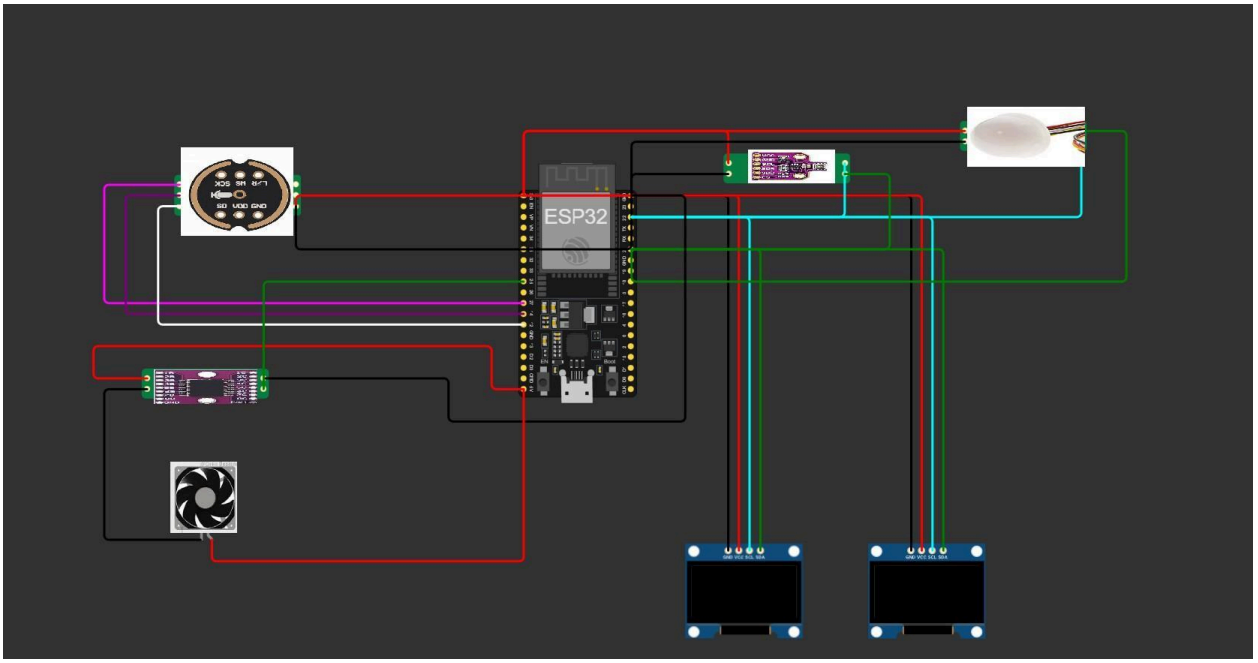
Quando il pin è attivo la ventola si accende.

Collegamento secondo display

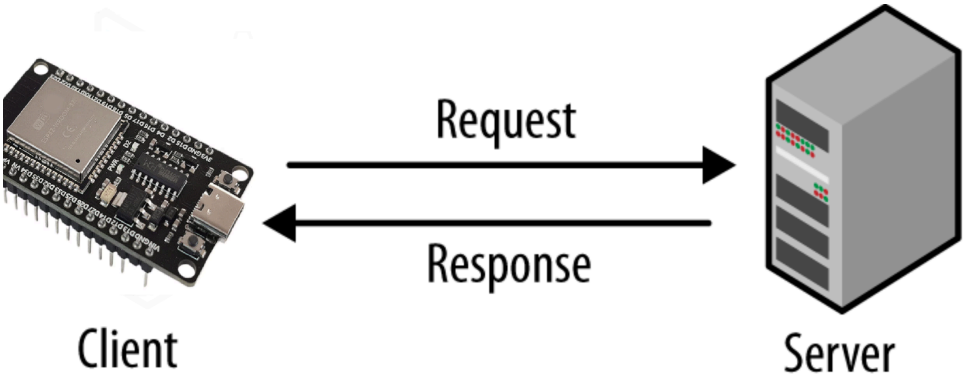
Il secondo display utilizza un **secondo bus I2C** per evitare conflitti.

Collegamenti:

SDA → GPIO 32
SCL → GPIO 33



7. Comunicazione con il server



Il dispositivo invia periodicamente i dati a un **server web tramite API REST**.

Questo permette di:

- salvare i dati
- visualizzarli su una dashboard
- controllare la ventola da remoto

I dati vengono inviati in formato **JSON**.

Esempio di dati inviati:

```
{
```

```
  "device_name": "esp32_living_room",
  "temperature": 23.4,
  "humidity": 41.2,
  "air_pressure": 1012.5,
  "gas_kohm": 75.3,
  "light_lux": 340,
  "noise_db": 42.5,
  "fan_state": false
}
```

Il server può rispondere indicando se la ventola deve essere accesa o spenta.

Response `201`

```json
{
  "telemetry": {
    "id": 42,
    "device_name": "esp32_living_room",
    "temperature": 32.5,
    "humidity": 61.3,
    "air_pressure": 1012.4,
    "gas_kohm": 118.7,
    "light_lux": 340.0,
    "noise_db": 49.2,
    "fan_state": false,
    "created_at": "2026-02-23T10:30:00.000000Z",
    "updated_at": "2026-02-23T10:30:00.000000Z"
  },
  "fan_desired_state": true
```

```
}
```

8. Struttura di uno Sketch Arduino

Uno sketch Arduino è un programma scritto in linguaggio **C/C++ semplificato**.

Ogni programma contiene sempre almeno due funzioni fondamentali.

Funzione setup()

La funzione `setup()` viene eseguita **una sola volta quando il dispositivo si accende**.

Serve per inizializzare i componenti.

Esempio dal progetto:

```
void setup() {


  Serial.begin(115200);

  Wire.begin(21, 22);


  pinMode(FAN_PIN, OUTPUT);

  digitalWrite(FAN_PIN, LOW);


}
```

Spiegazione

`Serial.begin(115200);`

avvia la comunicazione seriale con il computer per mostrare informazioni nel **monitor seriale**.

`Wire.begin(21,22);`

inizializza il **bus I2C principale** collegato ai pin:

- SDA → 21
- SCL → 22

`pinMode(FAN_PIN, OUTPUT);`

imposta il pin che controlla la ventola come **uscita digitale**.

Inizializzazione dei sensori

Sempre nella funzione `setup()` vengono inizializzati i sensori.

Esempio dal codice:

```
if (!bme.begin(0x77)) {
  Serial.println("BME680 non trovato");
  while (1);
}
```

Spiegazione

`bme.begin(0x77)`

inizializza il sensore BME680 usando l'indirizzo **0x77** sul bus I2C.

Se il sensore non viene trovato il programma si blocca.

Configurazione del microfono I2S

Il microfono viene configurato con questo blocco di codice:

```
i2s_config_t i2s_config = {
  .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
  .sample_rate = 16000,
  .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT
};
```

Spiegazione

Questo blocco definisce come deve funzionare il sistema audio:

- modalità ricezione
- frequenza di campionamento
- dimensione dei dati audio

Connessione al WiFi

Il dispositivo si connette alla rete con questo codice:

```
WiFi.begin(ssid, password);
```

```
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
}
```

Spiegazione

`WiFi.begin()`
inizia la connessione alla rete WiFi.

Il ciclo `while` continua finché il dispositivo non riesce a connettersi.

Funzione loop()

La funzione `loop()` è il cuore del programma.

Viene eseguita **continuamente**.

Esempio dal progetto:

```
void loop() {

  bme.performReading();
  float temp = bme.temperature;
  float hum  = bme.humidity;

}
```

Spiegazione

`bme.performReading()`

dice al sensore di effettuare una nuova misurazione.

Poi i valori vengono salvati nelle variabili:

- temperatura
- umidità

Controllo della ventola

La ventola si attiva automaticamente se la temperatura è troppo alta.

Codice:

```
bool fanOn = temp > 28.0;
```

```
digitalWrite(FAN_PIN, fanOn);
```

Spiegazione

Se la temperatura supera **28°C**, la variabile `fanOn` diventa vera.

Il pin che controlla la ventola viene quindi attivato.

Aggiornamento del display

I dati vengono mostrati sul display principale.

Codice:

```
display.setCursor(0,0);
```

```
display.print("Temp: ");
```

```
display.print(temp);
```

```
display.println(" C");
```

Spiegazione

`setCursor`
imposta la posizione del testo sullo schermo.

`print`
scrive il testo o i valori sul display.

Lettura del microfono

Il livello del suono viene calcolato con questo codice:

```
i2s_read(I2S_NUM_0, audioBuffer, sizeof(audioBuffer), &bytesRead,
portMAX_DELAY);
```

Spiegazione

Questa funzione legge i dati audio provenienti dal microfono e li salva nel buffer `audioBuffer.`

Successivamente il programma calcola il livello medio del segnale.

Visualizzazione data e ora

Il secondo display mostra data e ora.

Codice:

```
display.printf("%02d:%02d",
                timeinfo.tm_hour,
                timeinfo.tm_min);
```

Spiegazione

Il codice prende l’orario sincronizzato tramite internet e lo mostra sul display.